Universidad de los Andes > FACULTAD DE INGENIERÍA Y CIENCIAS APLICADAS

Arquitectura de Computadores
Proyecto N°2

Grupo: 5

Integrantes:

Juan Pablo Berríos

Agustin Flores

Ana Lobo

Profesor:

Jorge Gómez

13 de septiembre de 2026

ÍNDICE

ÍNDICE 2
1. Introducción 1
2. Arquitectura e Implementación del Sistema 2
2.1. Funcionamiento del Juego 2
2.2. Implementación en RTL 3
2.3. Assembly 3
2.3.1. Temporización y Generación pseudoaleatoria 4
2.3.2. Cálculos Aritméticos 4
2.3.3. Conversión a Decimal Codificado en Binario (BCD) 4
2.3.4. Robustez y Máquina de Estados 5
2.4. Assembler 5
2.4.1. Funcionamiento general 5
2.4.2. Sintaxis aceptada 7
2.4.3. Instrucciones utilizadas por el juego 8
2.4.4. Tratamiento de etiquetas 8
2.4.5. Tratamiento de inmediatos 9
2.4.5. Ejemplos de traducción 9
2.4.6. Pruebas utilizadas 10
3. Resultados y Conclusión 12
4. ANEXO 13
codigo: test_assembler.py 13
codigo: test_game.py 18

1. Introducción

La historia de la computación, ha sido una búsqueda constante por construir máquinas cada vez más capaces de ejecutar instrucciones automatizadas. Esta evolución permitió pasar de máquinas simples, diseñadas para realizar tareas específicas, a sistemas programables en los que el hardware y software trabaja en conjunto para resolver distintos tipos de problemas.

En la arquitectura de computadoras moderna, los procesadores son capaces de ejecutar programas almacenados en memoria e interactuar con distintos componentes del sistema. elementos como el conjunto de instrucciones, la memoria y periféricos determina cómo el software puede utilizar los recursos disponibles en el hardware. Asimismo, mecanismos como Memory-Mapped I/O (MMIO) permiten integrar los dispositivos de entrada y salida en el espacio de direcciones del procesador, posibilitando su control mediante instrucciones de lectura y escritura.

El presente proyecto tiene como objetivo profundizar en esta interacción mediante la implementación de un juego de reflejos sobre la arquitectura de Pochoco SoC, utilizando el procesador Espino Core y una FPGA Lattice iCE40 HX1K. Para ello, se agrega sobre la arquitectura existente, un contador de ciclos accesible desde software, el funcionamiento del juego en lenguaje Assembly y se implementa un assembler propio capaz de traducir dicho programa a código máquina. Finalmente, la jugabilidad conecta LEDs y botones para la interacción con el jugador.

En general, el proyecto integra distintas etapas de la arquitectura de computadores, como modificación y síntesis del hardware, programación de bajo nivel y su traducción a instrucciones ejecutables. Y así, comprender de manera práctica cómo una instrucción escrita en Assembly se transforma en código máquina, es ejecutada por el procesador e interactúa finalmente con periféricos físicos implementados sobre la FPGA.

2. Arquitectura e Implementación del Sistema

A continuación, se presentarán los diseños digitales creados para este proyecto, junto a su programación para la implementación en el hardware.

Para la implementación del juego se utilizó como base el Pochoco SoC, que conecta el procesador Espino Core a una memoria RAM unificada y distintos periféricos discos de la placa (como LEDs, interruptores y pantallas) y a una interfaz externa SPI esclava. Sobre esta arquitectura se realizaron modificaciones que permiten medir el tiempo directamente a partir de los ciclos del procesador y utilizar esta información desde el programa escrito en assembly.

2.1. Funcionamiento del Juego

El sistema implementa un juego de reflejos, donde el jugador debe reaccionar lo más rápido posible al encendido de uno de los LEDs de la placa, apretando los botones correspondientes, y utilizando los displays de siete segmentos para mostrar los tiempos obtenidos.

Al comienzo de cada ronda, los cuatro LEDs se mantienen encendidos durante al menos tres segundos. Luego, el sistema selecciona de manera pseudoaleatoria un LED como objetivo, y en ese momento comienza la medición del tiempo de respuesta del jugador. A continuación, el sistema espera hasta detectar la pulsación de un botón. Si el botón presionado corresponde al LED objetivo, ronda se considera como correcta, y se calcula el tiempo de respuesta en décimas de segundo.

El proceso se repite hasta completar diez respuestas correctas, y se van acumulando los tiempos obtenidos de todas las rondas. Al alcanzar la última respuesta correcta, se calcula el tiempo promedio de reacción entre las diez rondas y se presentan en los displays. En caso de que el jugador haga una jugada errónea, se reinicia la ronda actual, y esta no se considera entre las diez respuestas correctas.

2.2. Implementación en RTL

Se creó el módulo *game_top* como el módulo superior del diseño. Su función es instanciar el *pochoco_soc* manteniendo las interfaces físicas originales de la placa (LEDs, botones, displays de siete segmento). El beneficio de utilizarlo como módulo superior es que permite especificar el archivo que contiene el programa del juego mediante el parámetro MemFile, y de esta forma inicializar su memoria con el código generado a partir del programa *game.s*.

Además se agregó como periférico el contador de ciclos con memoria 0x8000000C. Al tratarse de periféricos mapeados en memoria, el programa puede interactuar con ellos mediante las mismas instrucciones de carga y almacenamiento utilizadas para acceder a memoria. Por ejemplo, una instrucción lw permite leer los botones o el contador, mientras que sw permite modificar los LEDs o los displays.

Para efectos de optimización en el uso de LUTs en el hardware, se dejaron comentadas en el código las interfaces y funcionalidades del SPI Slave, efectivamente eliminándolo de la implementación actual para el juego.

Como última medida de optimización a la hora de trabajar con la FPGA y cargar los programas a esta, se cambio la parte de ‘Power-on reset’ en el archivo ‘pochoco_soc.v’, para darle más tiempo a la FPGA de leer las instrucciones del hexadecimal y sin sacrificar tiempo aparente de carga para la persona que realiza la carga.

2.3. Assembly

El desarrollo del software se realizó bajo las estrictas limitaciones de la arquitectura RV32E del *Espino Core*, la cual restringe el banco registros a 16 (x0 a x15). Para evitar colisiones de datos y desbordamientos en memoria durante la ejecución, se estableció la siguiente convención:

- Se aislaron registros globales que mantiene su estado durante toda la partida, tales como x1 (contador de rondas) y x8 (acumulador del tiempo total en binario).
- Se designaron registros de acceso exclusivo a hardware (MMIO): x2 como puntero base, x14 de uso estricto para lectura de botones, y x13 y x15 para la captura de tiempos de reacción crudos.
- Para el manejo de datos y aritmética, se reciclaron registros dinámicamente según la etapa del ciclo de instrucción. Por ejemplo, x9 actúa como variable para constantes grandes durante la temporización, y luego se recicla como contador de decenas durante la conversión BCD.

2.3.1. Temporización y Generación pseudoaleatoria

La lógica de espera inicial de 3 segundos se implementó utilizando el *Cycle Counter* del procesador. Operando a una frecuencia de 25 Mhz, se calculó una constante de espera de 75.000.000 ciclos (0x047868C0), la cual se carga en memoria dividida en dos instrucciones (*lui* y *addi*) y se compara iterativamente contra la diferencia del reloj actual.

Para cumplir con el requisito de selección pseudoaleatoria del LED objetivo sin recurrir a instrucciones complejas, se diseñó un generador basado en entropía humana. Se extrae el valor instantáneo del *Cycle Counter* y se aplica una operación lógica XOR (*xor x11, x11, x15*) con el tiempo de reacción de la ronda anterior. Al enmascarar el resultado con un AND lógico (*andi x11, x11, 3*), se obtienen los dos bits menos significativos, garantizando una distribución de valores impredecible entre 0 y 3.

2.3.2. Cálculos Aritméticos

Dado que la unidad lógico aritmética (ALU) del SoC no implementa un multiplicador ni divisor por hardware, la conversión del tiempo de reacción crudo a décimas de segundo se resolvió mediante un algoritmo de división por restas sucesivas.

Sabiendo que 1 décima de segundo equivale a 2.500.000 ciclos de reloj, el algoritmo carga este valor (0x002625A0) y lo sustrae iterativamente del tiempo de reacción total, sumando una unidad al cociente (décimas) en cada ciclo hasta que el residuo sea menor al divisor. Este mismo principio se reutiliza al final de la partida para calcular el promedio exacto, dividiendo el acumulador global x8 por 10.

2.3.3. Conversión a Decimal Codificado en Binario (BCD)

Para que el display de 7 segmentos representara visualmente los números en base 10, fue necesario implementar un módulo de conversión a BCD.

Las decenas se extraen mediante restas sucesivas de 10. Posteriormente, estas decenas deben desplazarse 4 bits a la izquierda para conformar el *nibble* superior. Como las instrucciones de desplazamiento lógico de bits (*shifts*) se encuentran deshabilitadas en la síntesis del procesador para ahorrar LUTs, el desplazamiento se simuló matemáticamente multiplicando el valor por 16 mediante cuatro sumas consecutivas del registro sobre sí mismo (*add x9, x9, x9*).

2.3.4. Robustez y Máquina de Estados

El flujo del programa cuenta con manejo de errores (encendido de displays en par 'EE' al presionar un botón incorrecto sin avanzar el contador de rondas) y un sistema de control de rebotes manual al finalizar el juego. Antes de permitir un nuevo ciclo de ejecución, el programa entra en un bucle de bloqueo (*esperar_soltar_final*) que exige que todos los botones marquen 0 lógico, previniendo que una pulsación prolongada reinicie el sistema accidentalmente.

Una vez liberados, cualquier pulsación posterior redirige el puntero de instrucción a *_start*, reiniciando los acumuladores globales sin requerir un reset físico del hardware

2.4. Assembler

Para traducir el programa del juego a código máquina se desarrolló un assembler en Python (Version 3.14.7) en un archivo llamado "assembler.py". El programa recibe un archivo en assembly y traduce cada línea a una palabra de 32 bits, guardando dicha traducción en un archivo .hex, formato que la síntesis utiliza para inicializar la memoria en el SoC. El assembler solo utiliza librerías estándar de Python.

El comando de terminal en la ruta del repositorio para lograr la traducción es el siguiente:

```
python3 assembler.py sw/game.s -o sw/game.hex
```

Dónde *sw/game.s* es el archivo en assembly a traducir y *sw/game.hex* es el archivo hexadecimal donde se escribe la traducción.

La codificación sigue la ISA RV32E, que es la implementada por el core Espino del SoC Pochoco. Los valores de opcode, funct3 y funct7 de cada instrucción se obtuvieron directamente del decodificador del core "**rtl/espino_core/espino_decoder.v**", de modo que el assembler genera exactamente los campos que el hardware espera leer.

2.4.1. Funcionamiento general

El assembler opera en **dos pasadas** sobre el código fuente. Esto es necesario porque una instrucción de salto puede referirse a una etiqueta definida más adelante en el archivo. En

ese momento su dirección todavía no se conoce, ubicando primero todas las etiquetas y recién después codificar las instrucciones.

- **Lectura y limpieza:** Se lee el archivo línea por línea y se conserva el número de cada línea para poder reportar errores con su ubicación exacta. De cada línea se eliminan los comentarios y los espacios sobrantes.
- **Primera pasada:** Se recorren las líneas limpias manteniendo un contador de programa (PC) que comienza en `0x00000000`, la primera dirección del core. Cada etiqueta se registra en un diccionario que asocia su nombre con la dirección de la instrucción siguiente (en bytes). Cada instrucción se almacena junto con su PC, su mnemónico y su lista de operandos, donde el PC avanza en pasos de 4 bytes por instrucción.
- **Segunda pasada:** Para cada instrucción almacenada se expanden las pseudo-instrucciones, se busca el mnemónico en una tabla que indica su formato y sus campos fijos (opcode, funct3 y funct7), se interpretan los operandos según el formato, se valida el rango de los inmediatos y se construye la palabra de 32 bits ubicando cada campo en su posición.
- **Escritura:** Las palabras se escriben en el archivo de salida como 8 dígitos hexadecimales por línea. Si se detecta cualquier error, el assembler muestra el número de línea y la causa, termina con código de error y no genera el archivo de salida

2.4.2. Sintaxis aceptada

- Una instrucción por línea, opcionalmente precedida por una etiqueta:
  - Ejemplo: loop: sw x1, 0(x2)
  - Una etiqueta también puede ir sola en su línea.
- Los comentarios comienzan con '#' y se extienden hasta el final de la línea.
- Los mnemónicos no distinguen mayúsculas de minúsculas.
- Los operandos se separan por comas, con espacios opcionales.
- Los registros en el core Espino pertenecen al intervalo de x0…x15, cualquier valor fuera de este no se toma en consideración y suelta un error
- Se aceptan inmediatos en decimal, hexadecimal y binario
- Las Pseudo-instrucciones se traducen a instrucciones reales antes de codificar. Todas ocupan una sola instrucción:

Pseudo-instrucción	Traducción
j etiqueta	jal x0, etiqueta
mv rd, rs	addi rd, rs, 0

El assembler soporta las instrucciones RV32I de los formatos R, I, S, B, U y J que el core decodifica, exceptuando los desplazamientos, que fueron ignorados para el juego específico con el fin de ahorrar LUT´s en la FPGA

2.4.3. Instrucciones utilizadas por el juego

Instrucción	Formato	Opcode	Funct 3	Funct 7
add	R	0110011	000	0000000
sub	R	0110011	000	0100000
xor	R	0110011	100	0000000
addi	I	0010011	000	Na
andi	I	0010011	111	Na
lw	I	0000011	010	Na
sw	S	0100011	010	Na
beq	B	1100011	000	Na
bne	B	1100011	001	Na
bltu	B	1100011	110	Na
lui	U	0110111	Na	Na

2.4.4. Tratamiento de etiquetas

Las etiquetas se resuelven en la primera pasada mediante un diccionario. Se genera un error si una etiqueta está vacía o si se define dos veces, ya que en ese caso un salto a ella sería ambiguo. En la segunda pasada, cuando una instrucción de salto (formatos B y J) usa una etiqueta, el assembler calcula un desplazamiento relativo al PC de la propia instrucción de salto:

```
offset = dirección_de_la_etiqueta − PC_de_la_instrucción
```

2.4.5. Tratamiento de inmediatos

Cada inmediato se valida según el formato de la instrucción antes de codificarlo:

Formato	Número de Bits	Interpretación	Rango válido
I, S	12	con signo (+, -)	-2048…2047
U	20	sin signo (+)	0…0xFFFFF
B	13	con signo (+, -)	-4096…4095
J	21	con signo (+, -)	-2^{20}…(2^{20}-1)

Si el valor no pertenece dentro del intervalo válido, se genera un error y no se deja pasar silenciosamente.

Los valores negativos se representan en complemento a 2, y para conservar el número de bits especifico se utiliza el operador bitwise '&', juntando los bits del número y el hexadecimal correspondiente a su número de bits, ya que en Python los números no tienen un número de bits específico y rellena con 0s o 1s dependiendo de este.

En los formatos S, B y J, el inmediato no ocupa un solo bloque contiguo, sino que está repartido en varios campos de la instrucción, separando el acceso a los bits de este. Cada fragmento se extrae del inmediato y se ubica en la posición definida por la ISA, siguiendo exactamente el orden en que "espino_decoder.v" los reensambla.

2.4.5. Ejemplos de traducción

- **Instrucción tipo R**: (add x9, x12, x13)

Campo	funct 7	rs2	rs1	funct 3	rd	opcode
Bits	31-25	24-20	19-15	14-12	11-7	6-0
Valor	0	13	12	0	9	0x33
Binario	0000000	01101	01100	000	01001	0110011

```
def encode_r(opcode, f3, f7, rd, rs1, rs2):
    return (f7 << 25 | rs2 << 20 | rs1 << 15 | f3 << 12 | rd << 7 | opcode)
```

Resultado: 0000 0000 1101 0110 0000 0100 1011 0011 = 0x00d604b3

- **Salto de instrucción:** (bltu x10, x9, wait_3s).

La instrucción está en el “PC - 0x40” y la etiqueta wait_3s en “PC - 0x38”, así que se obtiene el valor del offset al restar estos valores (0x38 – 0x40 = -8 = offset)

En complemento a 2 de 13 bits, −8 es (1 1111 1111 1000) en binario y sus bits se encuentran repartidos en la instrucción según el formato B:

Campo	imm [12]	imm [10:5]	rs2	rs1	funct 3	imm [4:1]	imm [11]	opcode
Bits	31	30-25	24-20	19-15	14-12	11-8	7	6-0
Binario	1	111111	01001	01010	110	1100	1	1100011

```
def encode_b(opcode, f3, rs1, rs2, offset):
    return (((offset >> 12) & 0x1) << 31
            | ((offset >> 5) & 0x3F) << 25
            | rs2 << 20
            | rs1 << 15
            | f3 << 12
            | ((offset >> 1) & 0xF) << 8
            | ((offset >> 11) & 0x1) << 7
            | opcode)
```

Resultado: 1111 1110 1001 0101 0110 1100 1110 0011 = 0xfe956ce3

2.4.6. Pruebas utilizadas

Para poder probar la validez de ensamblador, se crearon 2 pruebas en Python, una en el archivo “test_assembler.py” y otra en “test_game.py”, las cuales ambas inidcan un mensaje “OK” si se logra pasar por completo.

La primera de las pruebas utiliza los programas ya escritos en la carpeta “sw” del proyecto, cada uno siendo un archivo .s con su respectivo .hex para comprar. Esta pasa el ensamblador por los 3 programas y compara el output de esta con los archivos hexadecimales ya existentes, además, esta prueba utiliza como referencia el ensamblador base que utiliza GNU.

La segunda prueba realiza un ensamblado del programa “game.s” que fue construido para el proyecto, en el cual se prueba que indique correctamente errores y compara con “assert” distintos valores teóricos con los obtenidos por el código, haciendo una traducción de ambas vías del código al archivo hexadecimal.

Para ver el código utilizado en ambas pruebas,puede ir a la carpeta “assembler_tests” del repositorio de github, así también como en el anexo al final de este documento..

3. Resultados y Conclusión

El desarrollo de este proyecto permitió concretar con éxito la integración entre hardware y el software de bajo nivel, implementando un juego de reacción funcional sobre la arquitectura del procesador Espino Core y el Pochoco SoC. A través de este trabajo, se comprendió de manera práctica el ciclo completo de ejecución de un programa, desde su escritura en lenguaje *Assembly* hasta su interacción física con los periféricos de la FPGA mediante *Memory-Mapped I/O* (MMIO).

Durante la programación del juego, las severas restricciones de la arquitectura RV32E, que cuenta con un banco de solo 16 registros, exigieron el diseño de una convención de memoria estricta y el reciclaje dinámico de registros para evitar colisiones de datos. Asimismo, la ausencia de operaciones de multiplicación y división por hardware, y la deshabilitación de las instrucciones de desplazamiento de bits (*shifts*), obligó a implementar soluciones algorítmicas eficientes.

El uso de restas sucesivas para la conversión de ciclos a décimas de segundo y el cálculo del promedio final, junto a la suma iterativa para lograr el desplazamiento en la conversión a BCD, demostraron un dominio profundo de la aritmética computacional a nivel de máquina.

Por otro lado, la construcción del *Assembler* propio en *Python* representó un hito fundamental para conectar el código fuente con el procesador físico. La implementación de un sistema de lectura en dos pasadas permitió resolver exitosamente las referencias adelantadas de las etiquetas y calcular los desplazamientos (*offsets*) relativos requeridos por las instrucciones de salto. Además, el manejo preciso a nivel de bits de los distintos formatos de instrucción (R, I, S, B, U, J), la validación de rangos y la correcta segmentación de los valores inmediatos garantizaron que el hardware recibiera el código máquina hexadecimal exacto para su decodificación.

En conclusión, este proyecto consolida los fundamentos de la arquitectura de computadores, evidenciando que el rendimiento y la funcionalidad de un sistema no solo depende de la síntesis del diseño físico (RTL), sino de la optimización lógica del software y la correcta traducción de las instrucciones para explotar al máximo los limitados recursos del hardware disponible.

4. ANEXO

codigo: test_assembler.py

```
import sys
from pathlib import Path

ROOT = Path(__file__).resolve().parent.parent   # la carpeta del repo
sys.path.insert(0, str(ROOT))                   # para que Python
encuentre assembler.py

from assembler import *

#
----------------------------------------------------------------------
----
# first_pass sobre game.s
#
----------------------------------------------------------------------
----
labels, instrs = first_pass(read_source("../sw/game.s"))

# 1) Chequeos fijos: dependen de la versión actual de game.s.
#    Si el juego cambia, actualizar estos números (y revisar que tengan
sentido).
assert len(instrs) == 97
assert labels["start_round"] == 0x30
assert labels["wait_3s"] == 0x4c
assert labels["show_target"] == 0x94
assert labels["correct"] == 0xb4
assert labels["game_over"] == 0x120
assert labels["wrong"] == 0x174

# 2) Chequeos generales: valen para cualquier versión de game.s.
pcs = [pc for _, pc, _, _ in instrs]
assert pcs == list(range(0, 4 * len(instrs), 4)), "los PC deben ser 0,
4, 8, ... sin saltos"
```

```
assert all(pc % 4 == 0 for pc in labels.values()), "toda etiqueta debe
estar alineada a 4"
assert all(pc ⩽ 4 * len(instrs) for pc in labels.values()), "ninguna
etiqueta fuera del programa"

# Toda etiqueta usada por un salto debe existir (si no, la pasada 2
fallará).
SALTOS = {"beq", "bne", "blt", "bge", "bltu", "bgeu", "jal", "j"}
for line_no, pc, mnem, ops in instrs:
    print(f"|| {line_no} || {pc} || {mnem} || {ops} ||")
    if mnem in SALTOS:
        destino = ops[-1]                  # la etiqueta siempre es el
último operando
        assert destino in labels, f"línea {line_no}: etiqueta
'{destino}' no definida"

print("ETIQUETAS")
for i, label in enumerate(labels):
    print(f"{i + 1} - {label} ||")

print("game.s OK:", len(instrs), "instrucciones,", len(labels),
"etiquetas")

# R
assert encode_r(0b0110011, 0b000, 0b0000000, rd=9, rs1=12, rs2=13) ==
0x00d604b3   # add x9, x12, x13   (lámina 20)
assert encode_r(0b0110011, 0b000, 0b0100000, rd=10, rs1=10, rs2=7) ==
0x40750533   # sub x10, x10, x7   (game.s) ← prueba funct7
assert encode_r(0b0110011, 0b100, 0b0000000, rd=11, rs1=11, rs2=15) ==
0x00f5c5b3  # xor x11, x11, x15  (game.s) ← prueba funct3
assert encode_r(0b0110011, 0b000, 0b0000000, rd=8, rs1=8, rs2=10)  ==
0x00a40433   # add x8, x8, x10    (game.s)
assert encode_r(0b0110011, 0b000, 0b0000000, rd=15, rs1=15, rs2=15) ==
0x00f787b3  # registro máximo de RV32E en los 3 campos
assert encode_r(0b0110011, 0b000, 0b0000000, rd=0, rs1=0, rs2=0)   ==
0x00000033   # todo en cero: solo queda el opcode
```

```
# I
assert encode_i(0b0010011, 0b000, rd=2, rs1=2, imm=4)      == 0x00410113
# addi x2, x2, 4     (blink.hex, línea 2)
assert encode_i(0b0010011, 0b000, rd=3, rs1=3, imm=-1)     == 0xfff18193
# addi x3, x3, -1    (blink.hex, línea 6)
assert encode_i(0b0010011, 0b100, rd=1, rs1=1, imm=15)     == 0x00f0c093
# xori x1, x1, 15    (blink.hex, línea 8)
assert encode_i(0b0010011, 0b111, rd=4, rs1=1, imm=15)     == 0x00f0f213
# andi x4, x1, 15    (7seg.hex, línea 8)
assert encode_i(0b0010011, 0b000, rd=9, rs1=9, imm=-1856) == 0x8c048493
# addi x9, x9, -1856 (game.s, los 3 segundos)
assert encode_i(0b0010011, 0b000, rd=1, rs1=0, imm=2047)   == 0x7ff00093
# máximo positivo de 12 bits
assert encode_i(0b0010011, 0b000, rd=1, rs1=0, imm=-2048) == 0x80000093
# mínimo negativo de 12 bits
assert encode_i(0b0000011, 0b010, rd=1, rs1=2, imm=0)      == 0x00012083
# lw x1, 0(x2)       (buttons_leds.hex, línea 5) ← load usa formato

# S
assert encode_s(0b0100011, 0b010, rs1=2, rs2=1, imm=0)      ==
0x00112023   # sw x1, 0(x2)    (blink.hex, línea 4)
assert encode_s(0b0100011, 0b010, rs1=3, rs2=1, imm=0)      ==
0x0011a023   # sw x1, 0(x3)    (buttons_leds.hex, línea 6)
assert encode_s(0b0100011, 0b010, rs1=4, rs2=7, imm=0)      ==
0x00722023   # sw x7, 0(x4)    (game.s, LEDs)
assert encode_s(0b0100011, 0b010, rs1=3, rs2=10, imm=0)     ==
0x00a1a023   # sw x10, 0(x3)   (game.s, display)
assert encode_s(0b0100011, 0b010, rs1=2, rs2=5, imm=-4)     ==
0xfe512e23   # sw x5, -4(x2)
assert encode_s(0b0100011, 0b010, rs1=2, rs2=5, imm=36)     ==
0x02512223   # sw x5, 36(x2)   ← usa ambos pedazos
assert encode_s(0b0100011, 0b010, rs1=2, rs2=5, imm=2047)   ==
0x7e512fa3   # máximo de 12 bits
assert encode_s(0b0100011, 0b010, rs1=2, rs2=5, imm=-2048) ==
0x80512023   # mínimo de 12 bits
assert encode_s(0b0100011, 0b000, rs1=2, rs2=5, imm=3)      ==
0x005101a3   # sb x5, 3(x2)    ← otro funct3
```

```
# U
assert encode_u(0b0110111, rd=2, imm=0x80000) == 0x80000137   # lui x2,
0x80000   (blink.hex, línea 1)
assert encode_u(0b0110111, rd=3, imm=0x200)   == 0x002001b7   # lui x3,
0x200     (blink.hex, línea 5)
assert encode_u(0b0110111, rd=3, imm=0x80000) == 0x800001b7   # lui x3,
0x80000   (buttons_leds.hex, línea 3)
assert encode_u(0b0110111, rd=9, imm=0x4787)  == 0x047874b7   # lui x9,
0x4787    (game.s, 75.000.000)
assert encode_u(0b0110111, rd=9, imm=0x262)   == 0x002624b7   # lui x9,
0x262     (game.s, 2.500.000)
assert encode_u(0b0110111, rd=1, imm=0xFFFFF) == 0xfffff0b7   # máximo
de 20 bits
assert encode_u(0b0110111, rd=5, imm=0)       == 0x000002b7   #
inmediato cero

# B
assert encode_b(0b1100011, 0b001, rs1=3, rs2=0, offset=-4)     ==
0xfe019ee3  # bne x3, x0, delay   (blink.hex, línea 7)
assert encode_b(0b1100011, 0b001, rs1=4, rs2=5, offset=8)      ==
0x00521463  # bne x4, x5, skip    (7seg.hex, línea 10)
assert encode_b(0b1100011, 0b001, rs1=1, rs2=5, offset=-40)    ==
0xfc509ce3  # bne x1, x5, loop    (7seg.hex, línea 13)
assert encode_b(0b1100011, 0b110, rs1=10, rs2=9, offset=-8)    ==
0xfe956ce3  # bltu x10, x9, wait_3s     (game.s)
assert encode_b(0b1100011, 0b000, rs1=14, rs2=0, offset=-4)    ==
0xfe070ee3  # beq x14, x0, wait_button (game.s)
assert encode_b(0b1100011, 0b000, rs1=14, rs2=12, offset=8)    ==
0x00c70463  # beq, adelante, chico
assert encode_b(0b1100011, 0b000, rs1=14, rs2=12, offset=-8)   ==
0xfec70ce3  # beq, atrás
assert encode_b(0b1100011, 0b000, rs1=14, rs2=12, offset=2048) ==
0x00c700e3  # solo offset[11] → instr[7]
assert encode_b(0b1100011, 0b000, rs1=14, rs2=12, offset=4094) ==
0x7ec70fe3  # máximo (+4094, par)
```

```
assert encode_b(0b1100011, 0b000, rs1=14, rs2=12, offset=-4096) ==
0x80c70063 # mínimo (-4096)

# J
assert encode_j(0b1101111, rd=0, offset=-20)      == 0xfedff06f   # jal
x0, loop   (blink.hex, línea 9)
assert encode_j(0b1101111, rd=0, offset=-48)      == 0xfd1ff06f   # jal
x0, loop   (7seg.hex, línea 15)
assert encode_j(0b1101111, rd=0, offset=-8)       == 0xff9ff06f   # jal
x0, loop   (buttons_leds.hex, línea 7)
assert encode_j(0b1101111, rd=0, offset=0x18)     == 0x0180006f   # jal
x0, show_target  (game.s, desde PC 0x68)
assert encode_j(0b1101111, rd=0, offset=8)        == 0x0080006f
assert encode_j(0b1101111, rd=1, offset=256)      == 0x100000ef   # jal
x1, ... ← rd ≠ 0 (llamada a subrutina)
assert encode_j(0b1101111, rd=0, offset=4096)     == 0x0000106f   #
solo offset[12]
assert encode_j(0b1101111, rd=0, offset=-128)     == 0xf81ff06f
assert encode_j(0b1101111, rd=0, offset=1048574)  == 0x7ffff06f   #
máximo (+1 MiB - 2)
assert encode_j(0b1101111, rd=0, offset=-1048576) == 0x8000006f   #
mínimo (-1 MiB)

for nombre in ["blink", "7seg", "buttons_leds"]:
    labels, instrs = first_pass(read_source(f"../sw/{nombre}.s"))
    words = second_pass(instrs, labels)
    esperado = [int(l, 16) for l in open(f"../sw/{nombre}.hex") if
l.strip()]
    for i, (w, e) in enumerate(zip(words, esperado)):
        if w ≠ e:
            print(f"{nombre} instrucción {i} (PC {4*i:#x}): obtuve
{w:08x}, esperaba {e:08x}")
    assert words == esperado, nombre
print("Los 3 programas coinciden con GNU as ✓")
```

codigo: test_game.py

```
"""
Test de ida y vuelta (round-trip) del assembler sobre sw/game.s.

No existe un game.hex "oficial" con qué comparar, así que este test
verifica
de otra forma: toma cada palabra que generó el assembler y la
DECODIFICA con
las mismas fórmulas que usa el procesador (espino_decoder.v, líneas
88-100, y
espino_id_stage.v, líneas 56-58). Si lo decodificado coincide con lo
que dice
game.s, el hardware va a ejecutar exactamente lo que escribimos.

El decodificador de aquí abajo es independiente de los encode_* del
assembler:
si un encoder tiene un bug, este test lo detecta.

Uso (desde la raíz del repo):
    python3 test_game.py
"""

import sys
from pathlib import Path

ROOT = Path(__file__).resolve().parent.parent   # la carpeta del repo
sys.path.insert(0, str(ROOT))                   # para que Python
encuentre assembler.py

from assembler import *

GAME = "../sw/game.s"
```

```
#
--------------------------------------------------------------------------
----
# Decodificador: copia en Python de lo que hace el hardware
#
--------------------------------------------------------------------------
----

def bits(w, hi, lo):
    """instr[hi:lo] en Verilog."""
    return (w >> lo) & ((1 << (hi - lo + 1)) - 1)


def sext(value, n_bits):
    """Extensión de signo: lo que hace {{20{instr[31]}}, ...} en el
decoder."""
    if value & (1 << (n_bits - 1)):
        return value - (1 << n_bits)
    return value


def decode(w):
    """Devuelve un dict con los campos de la instrucción, tal como los
ve el core."""
    d = {
        "opcode": bits(w, 6, 0),       # decoder.v línea 88
        "funct3": bits(w, 14, 12),     # línea 89
        "funct7": bits(w, 31, 25),     # línea 90
        "rd":     bits(w, 11, 7),      # id_stage.v línea 58
        "rs1":    bits(w, 19, 15),     # línea 56
        "rs2":    bits(w, 24, 20),     # línea 57
    }
    # Inmediatos: decoder.v líneas 94-100
    d["imm_i"] = sext(bits(w, 31, 20), 12)
    d["imm_s"] = sext((bits(w, 31, 25) << 5) | bits(w, 11, 7), 12)
    d["imm_b"] = sext((bits(w, 31, 31) << 12) | (bits(w, 7, 7) << 11)
```

```
                    | (bits(w, 30, 25) << 5) | (bits(w, 11, 8) << 1),
13)
    d["imm_u"] = bits(w, 31, 12)
    d["imm_j"] = sext((bits(w, 31, 31) << 20) | (bits(w, 19, 12) << 12)
                    | (bits(w, 20, 20) << 11) | (bits(w, 30, 21) <<
1), 21)
    return d


#
------------------------------------------------------------------------
----
# Helpers para leer los operandos del .s (independientes del assembler)
#
------------------------------------------------------------------------
----

def reg(tok):
    return int(tok.strip()[1:])


def mem(tok):
    off, r = tok.strip().rstrip(")").split("(")
    return int(off, 0), reg(r)


#
------------------------------------------------------------------------
----
# El test
#
------------------------------------------------------------------------
----

labels, instrs = first_pass(read_source(GAME))
words = second_pass(instrs, labels)
```

```
errores = []

def check(cond, line_no, pc, texto, msg):
    if not cond:
        errores.append(f"  línea {line_no:3d} (PC {pc:#05x})
{texto:<28}  → {msg}")


# 1) Una palabra por instrucción, y todas caben en 32 bits
assert len(words) == len(instrs), f"{len(words)} palabras para
{len(instrs)} instrucciones"
for w in words:
    assert 0 ⩽ w < 2**32, f"palabra fuera de 32 bits: {w:#x}"

# 2) Cada palabra, decodificada, dice lo mismo que el .s
for (line_no, pc, mnem_src, ops_src), w in zip(instrs, words):
    texto = f"{mnem_src} {', '.join(ops_src)}"
    mnem, ops = expand_pseudo(mnem_src, ops_src, line_no)
    tipo, opcode, f3, f7 = INSTRUCTIONS[mnem]
    d = decode(w)

    # opcode / funct3 / funct7 → ¿el hardware reconoce la instrucción
correcta?
    check(d["opcode"] == opcode, line_no, pc, texto, f"opcode
{d['opcode']:07b}, esperado {opcode:07b}")
    if f3 is not None:
        check(d["funct3"] == f3, line_no, pc, texto, f"funct3
{d['funct3']:03b}, esperado {f3:03b}")
    if f7 is not None:
        check(d["funct7"] == f7, line_no, pc, texto, f"funct7
{d['funct7']:07b}, esperado {f7:07b}")

    # Operandos según el tipo
    if tipo == "R":
        check(d["rd"] == reg(ops[0]),  line_no, pc, texto, f"rd =
x{d['rd']}")
```

```
        check(d["rs1"] == reg(ops[1]), line_no, pc, texto, f"rs1 =
x{d['rs1']}")
        check(d["rs2"] == reg(ops[2]), line_no, pc, texto, f"rs2 =
x{d['rs2']}")

    elif tipo == "I":
        check(d["rd"] == reg(ops[0]),  line_no, pc, texto, f"rd =
x{d['rd']}")
        check(d["rs1"] == reg(ops[1]), line_no, pc, texto, f"rs1 =
x{d['rs1']}")
        check(d["imm_i"] == int(ops[2], 0), line_no, pc, texto, f"imm =
{d['imm_i']}")

    elif tipo == "IL":
        off, base = mem(ops[1])
        check(d["rd"] == reg(ops[0]), line_no, pc, texto, f"rd =
x{d['rd']}")
        check(d["rs1"] == base,       line_no, pc, texto, f"rs1 =
x{d['rs1']}")
        check(d["imm_i"] == off,      line_no, pc, texto, f"offset =
{d['imm_i']}")

    elif tipo == "S":
        off, base = mem(ops[1])
        check(d["rs2"] == reg(ops[0]), line_no, pc, texto, f"rs2 =
x{d['rs2']}")
        check(d["rs1"] == base,        line_no, pc, texto, f"rs1 =
x{d['rs1']}")
        check(d["imm_s"] == off,       line_no, pc, texto, f"offset =
{d['imm_s']}")

    elif tipo == "B":
        destino = pc + d["imm_b"]   # a dónde salta REALMENTE el core
        check(d["rs1"] == reg(ops[0]), line_no, pc, texto, f"rs1 =
x{d['rs1']}")
        check(d["rs2"] == reg(ops[1]), line_no, pc, texto, f"rs2 =
x{d['rs2']}")
```

```
        check(destino == labels[ops[2]], line_no, pc, texto,
              f"salta a {destino:#x}, '{ops[2]}' está en
{labels[ops[2]]:#x}")

    elif tipo == "U":
        check(d["rd"] == reg(ops[0]), line_no, pc, texto, f"rd =
x{d['rd']}")
        check(d["imm_u"] == int(ops[1], 0), line_no, pc, texto, f"imm =
{d['imm_u']:#x}")

    elif tipo == "J":
        destino = pc + d["imm_j"]
        check(d["rd"] == reg(ops[0]), line_no, pc, texto, f"rd =
x{d['rd']}")
        check(destino == labels[ops[1]], line_no, pc, texto,
              f"salta a {destino:#x}, '{ops[1]}' está en
{labels[ops[1]]:#x}")

# 3) Ningún registro fuera de RV32E (el core ignora el bit 4 y x16 se
volvería x0)
for (line_no, pc, mnem, ops), w in zip(instrs, words):
    d = decode(w)
    for campo in ("rd", "rs1", "rs2"):
        tipo = INSTRUCTIONS[expand_pseudo(mnem, ops, line_no)[0]][0]
        usa = {"R": ("rd", "rs1", "rs2"), "I": ("rd", "rs1"), "IL":
("rd", "rs1"),
               "S": ("rs1", "rs2"), "B": ("rs1", "rs2"), "U": ("rd",),
"J": ("rd",)}[tipo]
        if campo in usa:
            check(d[campo] <= 15, line_no, pc, mnem, f"{campo} =
x{d[campo]} no existe en RV32E")


#
----------------------------------------------------------------------
----
# Resultado
```

```
#
--------------------------------------------------------------------------
----

if errores:
    print(f"game.s: {len(errores)} error(es) al decodificar:")
    print("\n".join(errores))
    raise SystemExit(1)

n_saltos = sum(1 for _, _, m, _ in instrs if m in {"beq", "bne", "blt",
"bge", "bltu", "bgeu", "jal", "j"})
print(f"game.s OK: {len(words)} instrucciones decodificadas
correctamente "
      f"({n_saltos} saltos llegan a su etiqueta)")
```